



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 12

“Proyecto 2: Bee Bot”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 24 de abril de 2026

INTRODUCCIÓN

Motorreductor

Un motorreductor es una unidad compacta y eficiente que integra un motor eléctrico y un tren de engranajes reductor. En la ingeniería de control y robótica, son la solución estándar cuando se requiere un movimiento preciso y con fuerza, sacrificando la alta velocidad nominal de los motores eléctricos convencionales.

El Principio de la Reducción

La función principal de un motorreductor se basa en la ley de conservación de la energía aplicada a la mecánica. Un motor de corriente continua (DC) común gira a velocidades muy altas (3,000 a 12,000 RPM) pero tiene un torque (par motor) casi insignificante.

Al acoplar un sistema de engranajes, ocurre una transformación:

- **Reducción de Velocidad:** La velocidad de salida es una fracción de la velocidad del motor.
- **Multiplicación de Torque:** La fuerza de giro en el eje final aumenta en la misma proporción que se reduce la velocidad (menos las pérdidas por fricción).

$$Torque_{salida} = Torque_{motor} * Relación_de_reducción * Eficiencia$$

Tipos de Engranajes (Arquitecturas)

La forma en que los engranajes están dispuestos dentro de la caja reductora determina el tamaño, la eficiencia y la fuerza del motorreductor:

- **Ejes Paralelos:** Los engranajes más comunes y económicos. Son eficientes, pero pueden ser ruidosos y no soportan torques extremadamente altos.
- **Planetarios:** Varios engranajes "planetas" giran alrededor de un engranaje "sol" central. Son muy compactos, soportan torques masivos y tienen una alta eficiencia.
- **Tornillo Sin Fin:** El eje del motor es un tornillo que mueve una corona. Su gran ventaja es que son irreversibles (el eje de salida no puede mover al motor), lo que actúa como un freno natural. Además, permiten una reducción muy alta en un espacio pequeño.

Especificaciones Críticas para Selección

Al elegir un motorreductor para un proyecto, hay que fijarse en estos parámetros técnicos:

- **Relación de Transmisión (Ratio):** Indica cuántas vueltas da el motor por cada vuelta del eje de salida (ej. 30:1, 100:1).

- **Torque de Puesto (Stall Torque):** Es el torque máximo que el motor genera cuando el eje está completamente detenido. Superar este límite puede romper los dientes de los engranajes.
- **Corriente de Operación y de Arranque:** Los motores demandan mucha más corriente al iniciar el movimiento o al enfrentar una carga pesada. Esto define el tipo de driver (como el L298N) que necesitarás.
- **Backlash:** Es el pequeño ángulo de libertad que tiene el eje de salida antes de que los engranajes se enganchen. En robótica de precisión, se buscan motores con "bajo backlash".

Puente H

Un Puente H es un circuito electrónico que permite a un microcontrolador controlar la dirección de giro de un motor de corriente continua (DC). Es una etapa de potencia esencial, ya que los pines de un microcontrolador no pueden entregar la corriente necesaria para mover un motor y, por sí solos, no pueden invertir la polaridad del voltaje.

Principio de Operación

El nombre proviene de la disposición de sus componentes (generalmente cuatro transistores o MOSFETs), que forman una estructura similar a la letra "H". El motor se ubica en la barra horizontal central.

- **Giro a la derecha:** Se activan dos interruptores en diagonal (ej. superior izquierdo e inferior derecho), permitiendo que la corriente fluya en un sentido.
- **Giro a la izquierda:** Se activan los otros dos interruptores en la diagonal opuesta, invirtiendo el sentido de la corriente y, por lo tanto, el giro del motor.
- **Frenado:** Si se activan los dos interruptores inferiores (o superiores) al mismo tiempo, el motor se cortocircuita y se detiene rápidamente.

Control de Velocidad

Además de la dirección, un Puente H permite controlar la velocidad mediante una señal PWM. Al encender y apagar los transistores miles de veces por segundo, el motor recibe un voltaje promedio que determina qué tan rápido gira.

Protecciones Necesarias

Un buen Puente H debe incluir Diodos de Libre Circulación (Flyback diodes). Cuando un motor se detiene, genera un pico de voltaje inverso que podría destruir los transistores del puente o incluso el microcontrolador si no hay estos diodos para disipar esa energía.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 12, el circuito a armar es el siguiente:

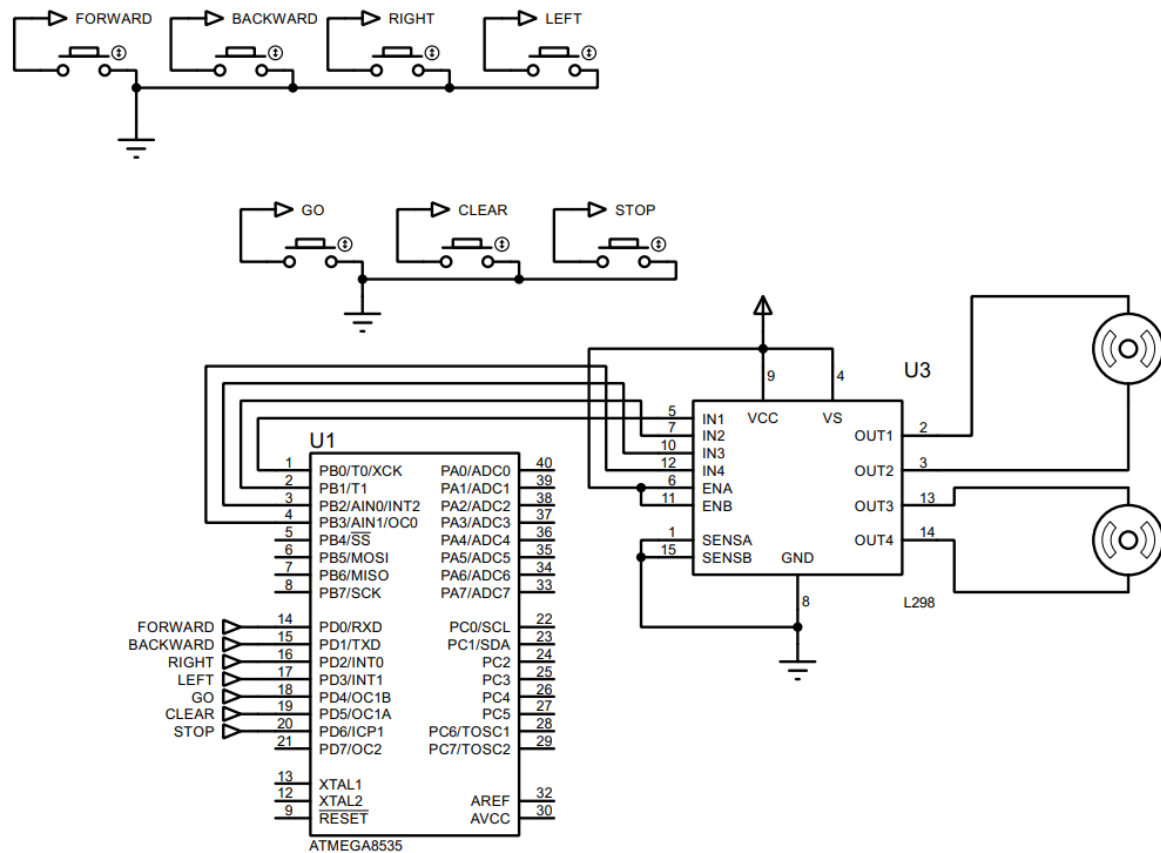
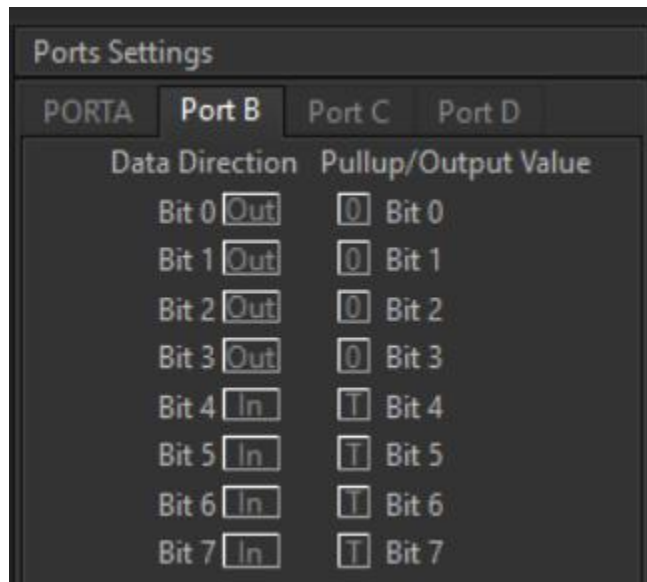


Imagen 1. Circuito creado en Proteus.

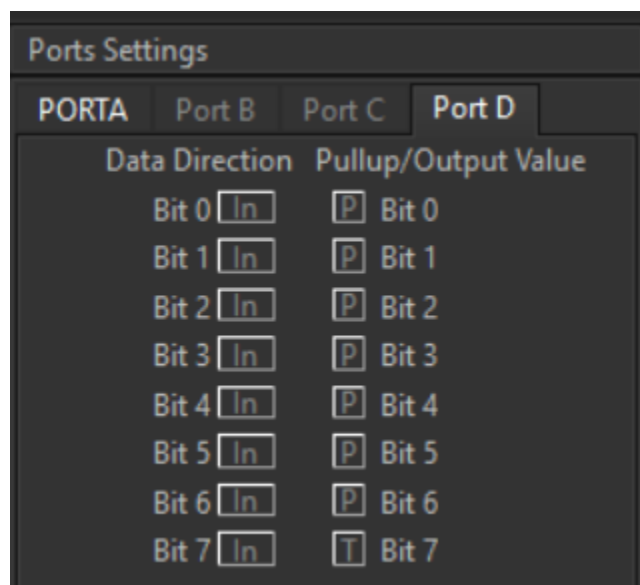
Configuración previa al código

De acuerdo con la imagen, el registro B (PB0-PB3) se configura como de salida, mientras que el registro D (PD0-PD6) se configura como de entrada, activando las resistencias de pull-up.



Ports Settings			
PORTA	Port B	Port C	Port D
Data Direction		Pullup/Output Value	
Bit 0	Out	0	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	In	T	Bit 4
Bit 5	In	T	Bit 5
Bit 6	In	T	Bit 6
Bit 7	In	T	Bit 7

Imagen 2. Configuraci3n del puerto B.



Ports Settings			
PORTA	Port B	Port C	Port D
Data Direction		Pullup/Output Value	
Bit 0	In	P	Bit 0
Bit 1	In	P	Bit 1
Bit 2	In	P	Bit 2
Bit 3	In	P	Bit 3
Bit 4	In	P	Bit 4
Bit 5	In	P	Bit 5
Bit 6	In	P	Bit 6
Bit 7	In	T	Bit 7

Imagen 3. Configuraci3n de puerto C.

C  digo del circuito de CodeVision:

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4       Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr  ctica 12 'Bee Bot'
8  Version : 1.0
9  Date    : 23/04/2026
10 Author  : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model         : Small
19 External RAM size    : 0
20 Data Stack size      : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <io.h>
26 #include <delay.h>
27
28 #define FORWARD_BTN PIND.0
29 #define BACKWARD_BTN PIND.1
30 #define RIGHT_BTN PIND.2
31 #define LEFT_BTN PIND.3
32 #define GO_BTN PIND.4
33 #define CLEAR_BTN PIND.5
34 #define STOP_BTN PIND.6

```

```

36 typedef enum {
37     STOP,
38     FORWARD,
39     BACKWARD,
40     RIGHT,
41     LEFT
42 } MovementDirection;
43
44 unsigned char btn_previous_state[7];
45 unsigned char i;
46 unsigned char btn_current_state[7];
47 MovementDirection movementSequence[10];
48 unsigned char sequencePosition = 0;
49 unsigned char programState = 0;
50 void updateButtonStates(void);
51 void recordMovements(void);
52 void clearMovementSequence(void);
53 void playMovementSequence(void);
54 void executeMovement(MovementDirection direction);
55
56 // Declare your global variables here
57
58 void main(void) {
59     // Declare your local variables here
60
61     // Input/Output Ports initialization
62     // Port A initialization
63     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
64     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
65     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
66     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

```

```

68 // Port B initialization
69 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=Out Bit2=Out Bit1=Out Bit0=Out
70 DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) | (1<<DDB0);
71 // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=0 Bit2=0 Bit1=0 Bit0=0
72 PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
73
74 // Port C initialization
75 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
76 DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
77 // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
78 PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
79
80 // Port D initialization
81 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
82 DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
83 // State: Bit7=T Bit6=P Bit5=P Bit4=P Bit3=P Bit2=P Bit1=P Bit0=P
84 PORTD=(0<<PORTD7) | (1<<PORTD6) | (1<<PORTD5) | (1<<PORTD4) | (1<<PORTD3) | (1<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);
85
86 // Timer/Counter 0 initialization
87 // Clock source: System Clock
88 // Clock value: Timer 0 Stopped
89 // Mode: Normal top=0xFF
90 // OC0 output: Disconnected
91 TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
92 TCNT0=0x00;
93 OCR0=0x00;

```

```

96 // Clock source: System Clock
97 // Clock value: Timer1 Stopped
98 // Mode: Normal top=0xFFFF
99 // OC1A output: Disconnected
100 // OC1B output: Disconnected
101 // Noise Canceler: Off
102 // Input Capture on Falling Edge
103 // Timer1 Overflow Interrupt: Off
104 // Input Capture Interrupt: Off
105 // Compare A Match Interrupt: Off
106 // Compare B Match Interrupt: Off
107 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
108 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
109 TCNT1H=0x00;
110 TCNT1L=0x00;
111 ICR1H=0x00;
112 ICR1L=0x00;
113 OCR1AH=0x00;
114 OCR1AL=0x00;
115 OCR1BH=0x00;
116 OCR1BL=0x00;
117
118 // Timer/Counter 2 initialization
119 // Clock source: System Clock
120 // Clock value: Timer2 Stopped
121 // Mode: Normal top=0xFF
122 // OC2 output: Disconnected
123 ASSR=0<<AS2;
124 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
125 TCNT2=0x00;
126 OCR2=0x00;

```



```

128 // Timer(s)/Counter(s) Interrupt(s) initialization
129 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
130
131 // External Interrupt(s) initialization
132 // INT0: Off
133 // INT1: Off
134 // INT2: Off
135 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
136 MCUCSR=(0<<ISC2);
137
138 // USART initialization
139 // USART disabled
140 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);
141
142 // Analog Comparator initialization
143 // Analog Comparator: Off
144 // The Analog Comparator's positive input is
145 // connected to the AIN0 pin
146 // The Analog Comparator's negative input is
147 // connected to the AIN1 pin
148 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
149 SFIOR=(0<<ACME);
150
151 // ADC initialization
152 // ADC disabled
153 ADSCRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
154
155 // SPI initialization
156 // SPI disabled
157 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);

```

```

159 // TWI initialization
160 // TWI disabled
161 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);
162
163 for (i = 0; i < 7; i++) {
164     btn_previous_state[i] = 1;
165     btn_current_state[i] = 1;
166 }
167
168 for (i = 0; i < 10; i++) {
169     movementSequence[i] = STOP;
170 }
171
172 while (1) {
173     // Place your code here
174     updateButtonStates();
175     switch (programState) {
176     case 0:
177         recordMovements();
178         break;
179     case 1:
180         clearMovementSequence();
181         programState = 0;
182         break;
183     case 2:
184         playMovementSequence();
185         programState = 0;
186         break;
187     case 4:
188         executeMovement(STOP);
189         programState = 0;
190         break;
191     default:
192         break;

```



```

193 |         }
194 |     }
195 |     for (i = 0; i < 7; i++) {
196 |         btn_previous_state[i] = btn_current_state[i];
197 |     }
198 | }
199 | }
200 |
201 | void updateButtonStates(void) {
202 |     btn_current_state[0] = FORWARD_BTN;
203 |     btn_current_state[1] = BACKWARD_BTN;
204 |     btn_current_state[2] = RIGHT_BTN;
205 |     btn_current_state[3] = LEFT_BTN;
206 |     btn_current_state[4] = GO_BTN;
207 |     btn_current_state[5] = CLEAR_BTN;
208 |     btn_current_state[6] = STOP_BTN;
209 | }
210 |
211 | void recordMovements(void) {
212 |     if (!btn_current_state[0] && btn_previous_state[0]) {
213 |         if (sequencePosition < 10) movementSequence[sequencePosition++] = FORWARD;
214 |         delay_ms(30);
215 |     } else if (!btn_current_state[1] && btn_previous_state[1]) {
216 |         if (sequencePosition < 10) movementSequence[sequencePosition++] = BACKWARD;
217 |         delay_ms(30);
218 |     } else if (!btn_current_state[2] && btn_previous_state[2]) {
219 |         if (sequencePosition < 10) movementSequence[sequencePosition++] = RIGHT;
220 |         delay_ms(30);
221 |     } else if (!btn_current_state[3] && btn_previous_state[3]) {
222 |         if (sequencePosition < 10) movementSequence[sequencePosition++] = LEFT;
223 |         delay_ms(30);
224 |     } else if (!btn_current_state[4] && btn_previous_state[4]) {
225 |         programState = 2;
226 |         delay_ms(30);
227 |     } else if (!btn_current_state[5] && btn_previous_state[5]) {
228 |         programState = 1;
229 |         delay_ms(30);
230 |     } else if (!btn_current_state[6] && btn_previous_state[6]) {
231 |         programState = 4;
232 |         delay_ms(30);
233 |     }
234 | }
235 |
236 | void clearMovementSequence(void) {
237 |     for(i = 0; i < 10; i++) {
238 |         movementSequence[i] = STOP;
239 |     }
240 |
241 |     sequencePosition = 0;
242 |     executeMovement(STOP);
243 | }
244 |
245 | void playMovementSequence(void) {
246 |     executeMovement(STOP);
247 |     delay_ms(500);
248 |
249 |     for(i = 0; i < sequencePosition; i++) {
250 |         if(!STOP_BTN) {
251 |             executeMovement(STOP);
252 |             return;
253 |         }
254 |         executeMovement(movementSequence[i]);
255 |     }
256 | }

```

```

258 void executeMovement(MovementDirection direction) {
259     unsigned int t;
260
261     switch(direction) {
262     case FORWARD:
263         PORTB = 0x05; // Adelante: (0101b) PA0=1, PA2=1
264         //delay_ms(2000);
265         break;
266     case BACKWARD:
267         PORTB = 0x0A; // Atr  s: (1010b) PA1=1, PA3=1
268         //delay_ms(2000);
269         break;
270     case RIGHT:
271         // Giro a la derecha (Motor Izquierdo Adelante 01, Motor Derecho Atr  s 10)
272         PORTB = 0x06; // (0110b) PA1=1, PA2=1
273         //delay_ms(2000);
274         break;
275     case LEFT:
276         // Giro a la izquierda (Motor Izquierdo Atr  s 10, Motor Derecho Adelante 01)
277         PORTB = 0x09; // (1001b) PA0=1, PA3=1
278         //delay_ms(2000);
279         break;
280     default:
281         PORTB = 0x00; // Detener: (0000b) Todos en 0
282         break;
283     }
284
285     for (t = 0; t < 200; t++) {
286         if (!STOP_BTN) {
287             PORTB = 0x00;
288             return;
289         }
290         delay_ms(10);
291     }
292
293     // Aseguramos que los motores se detengan brevemente tras la acci  n
294     PORTB = 0x00;
295     delay_ms(100);
296 }

```

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Este programa implementa un sistema de control para un robot, permite grabar y ejecutar secuencias de movimiento mediante los botones de direcciones y el almacenamiento en memoria. A través de una máquina de estados y estructuras como arreglos y enumeraciones, se logra una organización clara de las acciones del sistema. Además, se aplican conceptos fundamentales de sistemas embebidos como la detección de flancos, el control de salidas digitales y la secuenciación de instrucciones. Sin embargo, el uso de retardos bloqueantes y la limitada capacidad de almacenamiento representan áreas de mejora, ya que restringen la eficiencia y escalabilidad del sistema. Aun así, el programa demuestra de manera efectiva la integración entre hardware y software para el control de dispositivos físicos.

- **Mel3ndez Macedonio Rodrigo**

En el desarrollo de esta pr3ctica se comprob3 el funcionamiento pr3ctico de un sistema de control de motores a trav3s de la integraci3n de software y hardware, mejor conocido como Bee-Bot. Mediante la programaci3n del microcontrolador ATmega8535 y el uso del puente H L298, logramos traducir se3ales l3gicas y comandos almacenados en memoria en movimientos f3sicos precisos de los motorreductores.

El dise3o del circuito en Proteus y la posterior implementaci3n nos permitieron entender la importancia de las etapas de potencia. Los pines del microcontrolador por s3 solos no cuentan con la corriente necesaria para mover los motores, por lo que el puente H resulta indispensable para manejar la potencia requerida y permitir la inversi3n de polaridad para cambiar el sentido de giro. Asimismo, el uso de una m3quina de estados para registrar y reproducir la secuencia de movimientos demostr3 ser una soluci3n l3gica y estructurada para coordinar las acciones del Bee-Bot. Esta pr3ctica reafirma la relevancia de dise3ar interfaces de potencia seguras y de optimizar la l3gica de control para lograr sistemas embebidos confiables y funcionales.

BIBLIOGRAF3A

(s.f.). *Motorreductor*. Geek Electr3nica. Recuperado el 27 de abril de 2026, de <https://geekelectronica.com/>

(s.f.). *Puente H (electr3nica)*. Wikipedia. Recuperado el 27 de abril de 2026, de [https://es.wikipedia.org/wiki/Puente_H_\(electr%C3%B3nica\)](https://es.wikipedia.org/wiki/Puente_H_(electr%C3%B3nica))